# Clover Integration

## 1. Overview

### 1.1 Description

**Clover Integration** connects **Clover** (point-of-sale and merchant services platform) with **Newo Platform**.

It enables:

- Creating To-Go orders with menu items from Clover inventory
- Automatic inventory sync and caching (24-hour TTL)
- Order cancellation
- Order history retrieval and availability checking
- Customer lookup and creation with phone number matching
- SMS order confirmation notifications
- OAuth 2.0 authentication with automatic token refresh

This integration is designed for **restaurants, food service businesses, and retail merchants** who need **automated order taking through a conversational AI agent** (voice or chat).

### 1.2 Use Cases

- When a customer wants to place an order → AI extracts items from conversation, creates To-Go order in Clover POS
- When a customer asks about menu items → Agent references cached inventory with real-time prices
- When a customer wants to cancel an order → Delete the order from Clover
- When a customer asks about their orders → Retrieve order history within date range
- When a new customer places an order → Automatically create customer record in Clover with phone
- When a conversation starts → Sync inventory cache if outdated (24h default)

### 1.3 Supported Features

| Feature | Supported | Notes |
|---|---|---|
| Create Order | ✅ | Atomic order with line items from inventory |
| Cancel Order | ✅ | Deletes order from Clover |
| Order History | ✅ | By date range, optionally filtered by customer |
| Inventory Sync | ✅ | Cached with configurable TTL (default: 24h) |
| Customer Lookup | ✅ | By phone number (E.164) |
| Customer Creation | ✅ | Automatic during order if not found |
| Order Type Management | ✅ | Auto-creates "To-Go" type if not found |
| SMS Confirmation | ✅ | Sent after successful order creation |
| Concurrent Order Prevention | ✅ | Flag prevents duplicate orders |
| AI Payload Extraction | ✅ | LLM extracts items, quantities, dates from conversation |

| | | |
|---|---|---|
| OAuth 2.0 Token Management | ✅ | Auto-refresh with retry (up to 3 attempts) |
| JWT Merchant Extraction | ✅ | Merchant ID decoded from access token |
| Multi-Merchant Support | ❌ | One merchant per instance |
| Payment Processing | ❌ | Orders created with OPEN payment state |
| Appointment Scheduling | ❌ | POS ordering only, not calendar-based |

## 2. Requirements

Before installation:

- Active **Clover** merchant account
- **Clover OAuth App ID** and **App Secret**
- Clover API base URL (default: `https://apisandbox.dev.clover.com`)

## 3. How to Setup

### 3.1 Step 1 — Authorize in Clover

1. In Newo platform, set `clover_app_id` and `clover_app_secret`
2. Click the Clover authorization link provided by the platform
3. Log in to your Clover merchant account
4. Authorize data access for the Newo app
5. Copy the **authorization code** from the redirect URL
6. Paste the code into the `clover_oauth_code` attribute

### 3.2 Step 2 — Connect in Platform

1. Open **Newo → Projects**
2. Set the following attributes in **Builder / Attributes**:

| Attribute | Required | Description |
|---|---|---|
| `clover_app_id` | ✅ | OAuth App ID |
| `clover_app_secret` | ✅ | OAuth App Secret |
| `clover_oauth_code` | ✅ | One-time OAuth authorization code |
| `clover_base_url` | ❌ | API base URL (default: `https://apisandbox.dev.clover.com`) |
| `clover_order_type` | ❌ | Order type label (default: `To-Go`) |
| `clover_availability_days` | ❌ | Days for order history lookup (default: `1`) |
| `clover_static_data_update_interval` | ❌ | Inventory cache TTL in seconds (default: `86400`) |
| `clover_enable_slot_check` | ❌ | Enable order history checking (default: `True`) |

| | | |
|---|---|---|
| `clover_enable_booking` | ❌ | Enable order creation (default: `True`) |
| `clover_enable_cancellation` | ❌ | Enable order cancellation (default: `True`) |
| `clover_check_availability_on_conversation_start` | ❌ | Auto-check orders on session start (default: `False`) |

3. Click **Save + Publish All**
4. On publish, the SetupFlow automatically:
    - Exchanges the OAuth code for access and refresh tokens
    - Extracts merchant ID from JWT access token
    - Creates HTTP connectors for Clover API
    - Registers the order creation tool with the agent
    - Injects ordering payload schemas

## 4. How to Use

This section explains how the integration works, how to configure automation, and how to test it.

### 4.1 How the Integration Works

The integration uses Clover **Orders** as the primary entity and **Customers** for contact management. Inventory items are cached from the merchant's Clover catalog and matched against customer requests via LLM.

- A **Trigger** is a system event that starts a flow
- An **Action** is the API operation performed in Clover

**Available Triggers**

| Trigger | Event | Description |
|---|---|---|
| **Create Order** | `clover_create_order_tool` | When the agent confirms an order (tool call) |
| **Cancel Order** | `convo_agent_cancel_booking` | When the agent processes a cancellation |
| **Check Orders** | `get_available_slots` | When the agent looks up order history |
| **Session Start** | `session_started` | Syncs inventory cache, optional order preload |
| **Setup** | `project_publish_finish` | Initializes integration on deploy |

**Available Actions**

- **Get Items** — Fetch available inventory (GET `/v3/merchants/{id}/items`)
- **Get Order Types** — Fetch or create order types (GET/POST `/v3/merchants/{id}/order_types`)
- **Search Customer** — Find customer by phone (GET `/v3/merchants/{id}/customers`)
- **Create Customer** — New customer with phone, name, email (POST `/v3/merchants/{id}/customers`)
- **Create Order** — Atomic order with line items (POST `/v3/merchants/{id}/atomic_order/orders`)
- **Associate Customer** — Link customer to order (POST `/v3/merchants/{id}/orders/{id}`)
- **Delete Order** — Cancel an order (DELETE `/v3/merchants/{id}/orders/{id}`)

- **List Orders** — Order history by date range (GET `/v3/merchants/{id}/orders`)

### 4.2 Architecture Diagrams

#### Overall Sequence

```
sequenceDiagram
    participant U as User
    participant A as ConvoAgent
    participant I as Clover Integration
    participant T as Clover OAuth
    participant API as Clover API

    Note over I,T: Setup (on publish)
    A->>I: project_publish_finish
    I->>T: POST /oauth/v2/token (code)
    T-->>I: access_token + refresh_token
    I->>I: Decode JWT -> merchant_id
    I->>A: Order tool registered

    Note over U,API: Session Start (inventory sync)
    A->>I: session_started
    I->>I: Check cache TTL (24h default)
    alt Cache outdated
        I->>API: GET /v3/merchants/(id)/items
        API-->>I: Menu items
        I->>I: Cache inventory
    end

    Note over U,API: Order Creation
    U->>A: "I'd like 2 burgers and a coke"
    A->>I: clover_create_order_tool (memory)
    I->>I: LLM extract items, customer info, pickup time
    I->>API: GET /v3/merchants/(id)/order_types
    API-->>I: Order types
    I->>API: GET /v3/merchants/(id)/customers?filter=phone
    alt Customer not found
        I->>API: POST /v3/merchants/(id)/customers
        API-->>I: customer_id
    end
    I->>I: Match items to inventory
    I->>API: POST /v3/merchants/(id)/atomic_order/orders
    API-->>I: order_id
    I->>API: POST /v3/merchants/(id)/orders/(id) (customer)
    API-->>I: Associated
    I->>A: SMS confirmation + urgent message
    A->>U: "Your order is confirmed!"

    Note over U,API: Cancellation
    U->>A: "Cancel my order"
    A->>I: convo_agent_cancel_booking (booking_id)
    I->>API: DELETE /v3/merchants/(id)/orders/(id)
```

```
    API-->>I: Confirmation
    I->>A: result_success (cancelled)
    A->>U: "Your order has been cancelled"
```

**CreateOrderFlow**

```
flowchart TD
    E["clover_create_order_tool"] --> LOCK{"Order<br>in progress?"}
    LOCK -->|"Yes"| SKIP["Skip<br>(prevent duplicate)"]
    LOCK -->|"No"| SET["Set lock flag"]
    SET --> LLM["LLM extract from conversation:<br>items, quantities, customer info,
<br>pickup datetime"]
    LLM --> TYPE["Get/create<br>order type (To-Go)"]
    TYPE --> CUST["GET /customers<br>?filter=phone={phone}"]
    CUST --> FOUND{"Customer<br>found?"}
    FOUND -->|"No"| CREATE["POST /customers<br>{name, phone, email}"]
    FOUND -->|"Yes"| CID["Use customer_id"]
    CREATE --> CID
    CID --> MATCH["Match ordered items<br>to cached inventory"]
    MATCH --> ORDER["POST /atomic_order/orders<br>{lineItems, orderType,<br>title,
note, state}"]
    ORDER --> OK{"HTTP 200?"}
    OK -->|"No"| ERR["ResultError<br>(clear lock)"]
    OK -->|"Yes"| LINK["POST /orders/{id}<br>{customers, paymentState}"]
    LINK --> SMS["Send SMS confirmation<br>Send urgent message<br>Clear lock flag"]
```

**StaticDataFlow (Inventory Sync)**

```
flowchart TD
    E["session_started"] --> TOKEN{"Access token<br>exists?"}
    TOKEN -->|"No"| SKIP["Skip"]
    TOKEN -->|"Yes"| STALE{"Cache<br>outdated?"}
    STALE -->|"No"| SKIP
    STALE -->|"Yes"| API["GET /v3/merchants/{id}/items<br>?
limit=1000<br>&filter=available=true<br>&filter=hidden=false"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| PARSE["Extract per item:<br>id, name, price,<br>priceType,
unitName"]
    PARSE --> STORE["Store JSON in<br>clover_available_items<br>Update timestamp"]
```

**CancellationFlow**

```
flowchart TD
    E["convo_agent_cancel_booking"] --> GATE{"Cancellation<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| VAL{"booking_id<br>present?"}
    VAL -->|"No"| ERR["ResultError"]
    VAL -->|"Yes"| API["DELETE /v3/merchants/{id}<br>/orders/{booking_id}"]
```

```
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR
    OK -->|"Yes"| MSG["System message:<br>Order cancelled"]
```

**CheckOrdersFlow**

```
flowchart TD
    E["get_available_slots<br>or session_started"] --> GATE{"Order
check<br>enabled?"}
    GATE -->|"No"| SKIP["Skip"]
    GATE -->|"Yes"| RANGE["Calculate date range<br>(ms timestamps)"]
    RANGE --> API["GET /v3/merchants/{id}/orders<br>?filter=createdTime<br>(date
range)"]
    API --> OK{"HTTP 200?"}
    OK -->|"No"| ERR["ResultError"]
    OK -->|"Yes"| PARSE["Extract per order:<br>id, price, title,<br>state,
start_time"]
    PARSE --> OUT["result_success<br>{orders[]}"]
```

### 4.3 Where to Test

Testing can be done through the **Newo conversation interface** (chat or voice channel). Use the Clover Sandbox environment ( `https://apisandbox.dev.clover.com` ) for testing before switching to production.

### 4.4 Testing and Verification

**To test the integration:**

1. **Setup**: Publish and verify OAuth token exchange and merchant ID extraction
2. **Inventory**: Start a conversation and verify items are synced from Clover
3. **Order**: Tell the agent "I'd like to order [item from menu]" — verify order appears in Clover dashboard
4. **Customer**: Verify customer record created/found in Clover with correct phone
5. **Cancellation**: Cancel an order — verify deleted from Clover
6. **SMS**: Verify SMS confirmation sent after order creation

**If no action occurs:**

- Ensure OAuth credentials are set ( `clover_app_id` , `clover_app_secret` , `clover_oauth_code` )
- Verify `clover_access_token` was obtained (check it's not empty)
- Confirm `clover_merchant_id` was extracted from JWT
- Check that inventory was synced ( `clover_available_items` is not empty)
- Verify feature flags are enabled
- Ensure `clover_base_url` points to correct environment (sandbox vs production)

> *Note: This integration performs real operations in Clover POS. Orders created or deleted through the agent are reflected in the live merchant dashboard.*

## 5. FAQ

**Q: Does the integration handle payments?** A: No. Orders are created with `paymentState: OPEN`. Payment processing must be completed in the Clover POS or through a separate payment flow.

**Q: Can I connect multiple Clover merchants?** A: Each integration instance supports one merchant (extracted from JWT token). For multiple merchants, create separate integration instances.

**Q: How does the AI know which items to order?** A: Menu items are cached from the Clover inventory during session start. When a customer describes what they want, the LLM (Gemini) matches the request to available items by name, extracts quantities, and constructs the order payload.

**Q: How often is the inventory refreshed?** A: By default, every 24 hours (86400 seconds). Configure `clover_static_data_update_interval` to change the TTL. The cache refreshes automatically at the start of each conversation if outdated.

**Q: What happens if two orders are triggered simultaneously?** A: The integration uses a `clover_order_creation_in_progress` persona flag to prevent concurrent order creation. The second request is skipped until the first completes.

**Q: How are item quantities handled?** A: For PER_UNIT items (sold by weight/measure), `unitQty = quantity × 1000` (Clover's minor units). For fixed-price items, the line item is repeated in the array for each unit ordered.

**Q: What order type is used?** A: "To-Go" by default. If the order type doesn't exist in Clover, it's automatically created during the first order. The type ID is cached for subsequent orders.

**Q: Does the Sandbox work for testing?** A: Yes. Set `clover_base_url` to `https://apisandbox.dev.clover.com` for testing. Switch to the production URL when ready to go live.